SQL Interview Questions — Data Engineering

78 questions · try first, answers from page "Answers" · Sagar · Jan 2026

In the round

1. Ask about the grain, duplicates and NULLs before writing anything.
2. Say which dialect you're writing (Spark SQL, BigQuery, Postgres).
3. Write it step by step with CTEs, then mention edge cases and performance.

Basics & joins

1. **Second highest salary (return NULL if none).**
   emp(id, name, dept_id, salary, manager_id, hire_date)

2. **Nth highest salary.**
   emp(id, name, dept_id, salary, manager_id, hire_date)

3. **Employees who earn more than their manager.**
   emp(id, name, dept_id, salary, manager_id, hire_date)

4. **Departments with no employees.**
   dept(id, name), emp(id, name, dept_id, salary, manager_id, hire_date)

5. **Find duplicate emails.**
   person(id, email)

6. **Delete duplicates, keep the row with the smallest id.**
   person(id, email)

7. **Trap: row counts for INNER, LEFT, RIGHT, FULL join.**
   A(x) = 1,1,2,NULL B(x) = 1,1,NULL join on A.x = B.x

8. **Customers who never placed an order.**
   customers(customer_id, name), orders(order_id, customer_id, order_date, amount, status)

9. **Department with the highest average salary.**
   emp(id, name, dept_id, salary, manager_id, hire_date)

10. **Highest-paid employee(s) in each department.**
    emp(id, name, dept_id, salary, manager_id, hire_date)

11. **Employees hired in the last 90 days.**
    emp(id, name, dept_id, salary, manager_id, hire_date)

Sagar

12. **Swap 'M' and 'F' in one UPDATE.**
people(id, gender)

13. **Employees earning above the company average.**
emp(id, name, dept_id, salary, manager_id, hire_date)

14. **Managers with at least 5 direct reports.**
emp(id, name, dept_id, salary, manager_id, hire_date)

Aggregation

15. **Number of orders per customer, including customers with zero.**
customers(customer_id, name), orders(order_id, customer_id, order_date, amount, status)

16. **Total sales per month.**
orders(order_id, customer_id, order_date, amount, status)

17. **Each category's share of total sales (%).**
sales(category, amount)

18. **Customers who bought every product.**
purchases(customer_id, product_id), products(product_id)

19. **Customers who bought A and B but not C.**
purchases(customer_id, product)

20. **Pivot: sales per year with a column per quarter.**
sales(sale_date, amount)

21. **Unpivot: q1..q4 columns back into rows.**
quarterly(yr, q1, q2, q3, q4)

22. **Cities with more than 3 customers.**
customers(customer_id, city)

23. **Order count per status in a single row.**
orders(order_id, customer_id, order_date, amount, status)

24. **Median salary.**
emp(id, name, dept_id, salary, manager_id, hire_date)

Window functions

25. **Running total of sales by date.**
daily_sales(sale_date, amount)

Sagar

26. 7-day moving average.
daily_sales(sale_date, amount)

27. ROW_NUMBER vs RANK vs DENSE_RANK on salaries 100, 100, 90.
—

28. Top 3 salaries in each department.
emp(id, name, dept_id, salary, manager_id, hire_date)

29. Day-over-day change in revenue.
daily_sales(sale_date, amount)

30. Month-over-month growth %.
orders(order_id, customer_id, order_date, amount, status)

31. First and last order amount for each customer.
orders(order_id, customer_id, order_date, amount, status)

32. Each customer's second order.
orders(order_id, customer_id, order_date, amount, status)

33. Days between consecutive orders per customer.
orders(order_id, customer_id, order_date, amount, status)

34. Latest record per key (deduplicate CDC rows).
customer_updates(customer_id, city, updated_at)

35. Employees whose salary increased every year.
salary_history(emp_id, yr, salary)

36. Split customers into quartiles by total spend.
orders(order_id, customer_id, order_date, amount, status)

37. Top 10% of customers by spend.
orders(order_id, customer_id, order_date, amount, status)

38. Running total that resets every month.
daily_sales(sale_date, amount)

39. Compare each salary with its department average.
emp(id, name, dept_id, salary, manager_id, hire_date)

40. Rows where status changed from the previous row.
order_events(order_id, event_time, status)

Sagar

Dates, streaks & time series

41. **Users who logged in 3 or more consecutive days.**
logins(user_id, login_date)

42. **Longest login streak per user.**
logins(user_id, login_date)

43. **Find dates missing from a table.**
daily_sales(sale_date, amount)

44. **Users active last month but not this month (churned).**
activity(user_id, activity_date)

45. **Month-over-month retention: % of users who come back next month.**
activity(user_id, activity_date)

46. **Fraction of players who logged in again the day after their first login.**
activity(player_id, event_date)

47. **Sessionize clicks: a new session after 30 minutes of inactivity.**
clicks(user_id, ts)

48. **Find overlapping bookings for the same room.**
bookings(id, room_id, start_ts, end_ts)

49. **Collapse consecutive numbers into ranges.**
nums(n) = 1,2,3,5,6,9

50. **Active subscriptions on each day.**
subs(user_id, start_date, end_date), calendar(d)

Classic problems

51. **Numbers that appear at least 3 times in a row.**
logs(id, num) — ids are sequential

52. **Days warmer than the previous day.**
weather(id, record_date, temp)

53. **Cancellation rate per day for unbanned users.**
trips(id, client_id, driver_id, status, request_at), users(user_id, banned)

54. **Person with the most friends.**
friends(requester_id, accepter_id)

55. **Swap every two adjacent seats.**
seat(id, student) — ids 1..n

Sagar

56. Label each node as Root, Inner or Leaf.

tree(id, parent_id)

57. Click-through rate per ad, without division errors.

ad_events(ad_id, action) — 'click' / 'view' / 'ignore'

58. Product pairs most often bought together.

order_items(order_id, product_id)

59. Employee hierarchy: all reports under a manager, any level.

emp(id, name, dept_id, salary, manager_id, hire_date)

Data engineering SQL

60. Upsert new and changed rows into a target (SCD Type 1).

target(id, name, city), staging(id, name, city)

61. SCD Type 2: keep history when a customer's city changes.

dim_customer(id, city, start_date, end_date, is_current), staging(id, city)

62. Incremental load: only rows changed since the last load.

source(id, ..., updated_at), etl_watermark(table_name, last_loaded)

63. Rows in source missing from target, and rows that changed.

src(id, name, city), tgt(id, name, city)

64. Reconcile row counts between source and target per load date.

src(load_date, ...), tgt(load_date, ...)

65. Check staging for duplicate primary keys before loading.

staging(order_id, ...)

66. Null-count report for key columns in one pass.

customers(customer_id, email, phone, city)

67. Latest row per key using QUALIFY.

events(id, payload, updated_at)

68. Explode an array column into rows.

orders(order_id, items ARRAY<STRING>)

69. Why does this query scan the whole table? Fix it.

events partitioned by event_date
WHERE YEAR(event_date) = 2026

Sagar

Concepts & output traps

70. Logical order of execution of a SELECT.

—

71. WHERE vs HAVING.

—

72. Trap: COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col).

t(col) = 1, 1, NULL, 2

73. UNION vs UNION ALL.

—

74. Trap: why does WHERE city <> 'Delhi' miss some rows?

customers.city has NULLs

75. Trap: why does NOT IN return no rows?

SELECT * FROM a WHERE id NOT IN (SELECT id FROM b) — b.id has a NULL

76. DELETE vs TRUNCATE vs DROP.

—

77. CTE vs subquery vs temp table.

—

78. A query got slow. How do you debug it?

—

Sagar

Answers

Basics & joins

1. Second highest salary (return NULL if none).

```
SELECT MAX(salary) AS second_highest
FROM emp
WHERE salary < (SELECT MAX(salary) FROM emp);
```

▶ MAX on an empty set returns NULL, so the 'no second salary' case is handled for free.

2. Nth highest salary.

```
SELECT DISTINCT salary
FROM (SELECT salary,
             DENSE_RANK() OVER (ORDER BY salary DESC) AS r
      FROM emp) t
WHERE r = :n;
```

▶ DENSE_RANK so ties don't skip ranks. LIMIT/OFFSET also works but breaks on duplicates.

3. Employees who earn more than their manager.

```
SELECT e.name
FROM emp e
JOIN emp m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

▶ Self join: the same table plays employee and manager.

4. Departments with no employees.

```
SELECT d.name
FROM dept d
LEFT JOIN emp e ON e.dept_id = d.id
WHERE e.id IS NULL;
```

▶ LEFT JOIN + IS NULL = anti join. Filter on the right table's key, not a nullable column.

5. Find duplicate emails.

```
SELECT email, COUNT(*) AS cnt
FROM person
GROUP BY email
HAVING COUNT(*) > 1;
```

▶ HAVING filters groups; WHERE can't use aggregates.

Sagar

6. Delete duplicates, keep the row with the smallest id.

```
DELETE FROM person
WHERE id IN (
  SELECT id FROM (
    SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
    FROM person) t
  WHERE rn > 1);
```

▶ The extra derived table avoids MySQL's 'can't modify the table you select from' error.

7. Trap: row counts for INNER, LEFT, RIGHT, FULL join.

```
-- INNER = 4  (each 1 in A matches both 1s in B: 2 x 2)
-- LEFT  = 6  (4 + unmatched 2 + unmatched NULL)
-- RIGHT = 5  (4 + unmatched NULL in B)
-- FULL  = 7  (4 + 2 from A + 1 from B)
```

▶ Duplicates multiply. NULL never equals NULL, so NULLs never match. Very commonly asked.

8. Customers who never placed an order.

```
SELECT c.name
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o
                  WHERE o.customer_id = c.customer_id);
```

▶ Prefer NOT EXISTS over NOT IN — NOT IN returns nothing if the subquery has a NULL.

9. Department with the highest average salary.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM emp
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

▶ Ties? Use RANK() over the averages and keep rank 1.

10. Highest-paid employee(s) in each department.

```
SELECT dept_id, name, salary
FROM (SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS r
      FROM emp) t
WHERE r = 1;
```

▶ RANK keeps ties. Correlated subquery (salary = MAX per dept) also works but scans more.

Sagar

11. Employees hired in the last 90 days.

```
SELECT *
FROM emp
WHERE hire_date >= CURRENT_DATE - INTERVAL '90' DAY;
```

▶ Keep the function on the constant side so an index or partition on hire_date can be used.

12. Swap 'M' and 'F' in one UPDATE.

```
UPDATE people
SET gender = CASE gender WHEN 'M' THEN 'F'
                         WHEN 'F' THEN 'M' END;
```

▶ One statement, no temp value needed.

13. Employees earning above the company average.

```
SELECT name, salary
FROM emp
WHERE salary > (SELECT AVG(salary) FROM emp);
```

▶ Scalar subquery runs once.

14. Managers with at least 5 direct reports.

```
SELECT m.name
FROM emp e
JOIN emp m ON e.manager_id = m.id
GROUP BY m.id, m.name
HAVING COUNT(*) >= 5;
```

▶ Group by the manager's id, not just the name — names aren't unique.

Aggregation

15. Number of orders per customer, including customers with zero.

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.name;
```

▶ COUNT(o.order_id) ignores NULLs → 0. COUNT(*) would wrongly return 1.

Sagar

16. Total sales per month.

```
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS sales
FROM orders
GROUP BY 1
ORDER BY 1;
```

▶ Group by month start, not by MONTH() alone — that would merge Jan 2025 with Jan 2026.

17. Each category's share of total sales (%).

```
SELECT category,
       SUM(amount) AS sales,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct
FROM sales
GROUP BY category;
```

▶ Window over an aggregate: SUM(SUM(x)) OVER () = grand total. 100.0 avoids integer division.

18. Customers who bought every product.

```
SELECT customer_id
FROM purchases
GROUP BY customer_id
HAVING COUNT(DISTINCT product_id) = (SELECT COUNT(*) FROM products);
```

▶ Relational division. DISTINCT matters if a customer bought the same product twice.

19. Customers who bought A and B but not C.

```
SELECT customer_id
FROM purchases
GROUP BY customer_id
HAVING SUM(CASE WHEN product = 'A' THEN 1 ELSE 0 END) > 0
   AND SUM(CASE WHEN product = 'B' THEN 1 ELSE 0 END) > 0
   AND SUM(CASE WHEN product = 'C' THEN 1 ELSE 0 END) = 0;
```

▶ Conditional aggregation in HAVING — one scan, no self joins.

Sagar

20. Pivot: sales per year with a column per quarter.

```
SELECT EXTRACT(YEAR FROM sale_date) AS yr,
  SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 1 THEN amount ELSE 0 END) AS q1,
  SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 2 THEN amount ELSE 0 END) AS q2,
  SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 3 THEN amount ELSE 0 END) AS q3,
  SUM(CASE WHEN EXTRACT(QUARTER FROM sale_date) = 4 THEN amount ELSE 0 END) AS q4
FROM sales
GROUP BY 1;
```

▶ CASE pivot works in every database. Some engines also have a PIVOT keyword.

21. Unpivot: q1..q4 columns back into rows.

```
SELECT yr, 'Q1' AS qtr, q1 AS amount FROM quarterly
UNION ALL SELECT yr, 'Q2', q2 FROM quarterly
UNION ALL SELECT yr, 'Q3', q3 FROM quarterly
UNION ALL SELECT yr, 'Q4', q4 FROM quarterly;
```

▶ UNION ALL, not UNION — UNION would drop legitimately equal rows. Spark has stack(), others UNPIVOT.

22. Cities with more than 3 customers.

```
SELECT city, COUNT(*) AS customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 3;
```

▶ Simple, but they watch whether you use HAVING correctly.

23. Order count per status in a single row.

```
SELECT
  COUNT(CASE WHEN status = 'DELIVERED' THEN 1 END) AS delivered,
  COUNT(CASE WHEN status = 'CANCELLED' THEN 1 END) AS cancelled,
  COUNT(CASE WHEN status = 'RETURNED'  THEN 1 END) AS returned
FROM orders;
```

▶ COUNT ignores NULL, and CASE without ELSE returns NULL. FILTER (WHERE ...) is the Postgres form.

Sagar

24. Median salary.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median
FROM emp;
```

▶ Without PERCENTILE_CONT: ROW_NUMBER ascending and descending, keep rows where they're within 1 of each other, average them.

Window functions

25. Running total of sales by date.

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running
FROM daily_sales;
```

▶ Say ROWS explicitly. The default RANGE frame adds up all rows with the same date together.

26. 7-day moving average.

```
SELECT sale_date,
       AVG(amount) OVER (ORDER BY sale_date
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM daily_sales;
```

▶ Assumes one row per day. Missing days → fill a calendar first, or use a RANGE interval frame.

27. ROW_NUMBER vs RANK vs DENSE_RANK on salaries 100, 100, 90.

```
-- salary  ROW_NUMBER  RANK  DENSE_RANK
--  100        1        1       1
--  100        2        1       1
--   90        3        3       2
```

▶ ROW_NUMBER = unique, RANK = gaps after ties, DENSE_RANK = no gaps. Asked in almost every round.

28. Top 3 salaries in each department.

```
SELECT dept_id, name, salary
FROM (SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
r
      FROM emp) t
WHERE r <= 3;
```

▶ Clarify: top 3 distinct salaries (DENSE_RANK) or top 3 people (ROW_NUMBER)?

Sagar

29. Day-over-day change in revenue.

```
SELECT sale_date, amount,
       amount - LAG(amount) OVER (ORDER BY sale_date) AS change
FROM daily_sales;
```

▶ The first row is NULL — that's correct, not a bug.

30. Month-over-month growth %.

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS rev
  FROM orders GROUP BY 1)
SELECT month, rev,
       ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY month))
            / NULLIF(LAG(rev) OVER (ORDER BY month), 0), 2) AS growth_pct
FROM m;
```

▶ Aggregate first, then window. NULLIF prevents division-by-zero errors.

31. First and last order amount for each customer.

```
SELECT DISTINCT customer_id,
  FIRST_VALUE(amount) OVER w AS first_amt,
  LAST_VALUE(amount)  OVER w AS last_amt
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date
             ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

▶ Trap: LAST_VALUE with the default frame returns the current row. You need the full frame.

32. Each customer's second order.

```
SELECT *
FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY order_date, order_id) AS rn
      FROM orders) t
WHERE rn = 2;
```

▶ Add a tie-breaker (order_id) so the result is deterministic.

33. Days between consecutive orders per customer.

```
SELECT customer_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_id
                                          ORDER BY order_date) AS days_since_prev
FROM orders;
```

▶ Date subtraction syntax varies: DATEDIFF in Spark/MySQL, DATE_DIFF in BigQuery.

34. Latest record per key (deduplicate CDC rows).

```
SELECT customer_id, city, updated_at
FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id
                                   ORDER BY updated_at DESC) AS rn
      FROM customer_updates) t
WHERE rn = 1;
```

▶ The single most used data-engineering query. Spark/Databricks, Snowflake and BigQuery can use QUALIFY rn = 1.

35. Employees whose salary increased every year.

```
SELECT emp_id
FROM (SELECT emp_id, salary,
             LAG(salary) OVER (PARTITION BY emp_id ORDER BY yr) AS prev
      FROM salary_history) t
GROUP BY emp_id
HAVING SUM(CASE WHEN prev IS NOT NULL AND salary <= prev THEN 1 ELSE 0 END) = 0;
```

▶ Count the violations and keep employees with zero.

36. Split customers into quartiles by total spend.

```
SELECT customer_id, total,
       NTILE(4) OVER (ORDER BY total DESC) AS quartile
FROM (SELECT customer_id, SUM(amount) AS total
      FROM orders GROUP BY customer_id) t;
```

▶ NTILE gives buckets as equal in size as possible.

37. Top 10% of customers by spend.

```
SELECT customer_id, total
FROM (SELECT customer_id, SUM(amount) AS total,
             PERCENT_RANK() OVER (ORDER BY SUM(amount) DESC) AS pr
      FROM orders GROUP BY customer_id) t
WHERE pr <= 0.10;
```

▶ Window functions can wrap aggregates in the same SELECT.

38. Running total that resets every month.

```
SELECT sale_date, amount,
       SUM(amount) OVER (PARTITION BY DATE_TRUNC('month', sale_date)
                         ORDER BY sale_date) AS mtd
FROM daily_sales;
```

▶ Month-to-date = partition by month. Same idea for YTD with year.

39. Compare each salary with its department average.

```
SELECT name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff
FROM emp;
```

▶ Window keeps every row — no GROUP BY plus join back.

40. Rows where status changed from the previous row.

```
SELECT *
FROM (SELECT *, LAG(status) OVER (PARTITION BY order_id
                                  ORDER BY event_time) AS prev
      FROM order_events) t
WHERE prev IS NULL OR status <> prev;
```

▶ Removes repeated identical events and keeps only real transitions.

Dates, streaks & time series

41. Users who logged in 3 or more consecutive days.

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
g AS (SELECT user_id, login_date,
             login_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id
                                  ORDER BY login_date) AS INT) AS grp
      FROM d)
SELECT DISTINCT user_id
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

▶ Gaps and islands: date minus row number stays constant inside a streak. In Spark use DATE_SUB(login_date, rn).

42. Longest login streak per user.

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
g AS (SELECT user_id,
             login_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id
                                  ORDER BY login_date) AS INT) AS grp
      FROM d)
SELECT user_id, MAX(len) AS longest
FROM (SELECT user_id, grp, COUNT(*) AS len FROM g GROUP BY user_id, grp) s
GROUP BY user_id;
```

▶ Same island trick, then the max island size.

43. Find dates missing from a table.

```
WITH RECURSIVE cal AS (
  SELECT MIN(sale_date) AS d, MAX(sale_date) AS mx FROM daily_sales
  UNION ALL
  SELECT d + INTERVAL '1' DAY, mx FROM cal WHERE d < mx)
SELECT d FROM cal
WHERE d NOT IN (SELECT sale_date FROM daily_sales);
```

▶ Generate a calendar, then anti join. Spark: SEQUENCE + EXPLODE; BigQuery: GENERATE_DATE_ARRAY.

44. Users active last month but not this month (churned).

```
SELECT DISTINCT user_id FROM activity
WHERE activity_date >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1' MONTH
  AND activity_date <  DATE_TRUNC('month', CURRENT_DATE)
EXCEPT
SELECT DISTINCT user_id FROM activity
WHERE activity_date >= DATE_TRUNC('month', CURRENT_DATE);
```

▶ EXCEPT = set difference. Keep date ranges half-open (>= start, < next start).

45. Month-over-month retention: % of users who come back next month.

```
WITH m AS (SELECT DISTINCT user_id,
                  DATE_TRUNC('month', activity_date) AS month
           FROM activity)
SELECT a.month,
       COUNT(DISTINCT a.user_id) AS active,
       COUNT(DISTINCT b.user_id) AS retained,
       ROUND(100.0 * COUNT(DISTINCT b.user_id) / COUNT(DISTINCT a.user_id), 2) AS pct
FROM m a
LEFT JOIN m b ON b.user_id = a.user_id
             AND b.month = a.month + INTERVAL '1' MONTH
GROUP BY a.month;
```

▶ Self join on month + 1. Classic product-analytics question.

46. Fraction of players who logged in again the day after their first login.

```
WITH f AS (SELECT player_id, MIN(event_date) AS first_day
           FROM activity GROUP BY player_id)
SELECT ROUND(1.0 * COUNT(a.player_id) / COUNT(*), 2) AS fraction
FROM f
LEFT JOIN activity a ON a.player_id = f.player_id
                    AND a.event_date = f.first_day + INTERVAL '1' DAY;
```

▶ Day-1 retention. Count matches from the LEFT JOIN over all first-time players.

47. Sessionize clicks: a new session after 30 minutes of inactivity.

```
WITH x AS (
  SELECT user_id, ts,
         CASE WHEN LAG(ts) OVER (PARTITION BY user_id ORDER BY ts) IS NULL
                OR ts - LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)
                   > INTERVAL '30' MINUTE
              THEN 1 ELSE 0 END AS new_session
  FROM clicks)
SELECT user_id, ts,
       SUM(new_session) OVER (PARTITION BY user_id ORDER BY ts) AS session_id
FROM x;
```

▶ Flag the start of each session, then a running sum of flags numbers the sessions.

48. Find overlapping bookings for the same room.

```
SELECT a.id, b.id
FROM bookings a
JOIN bookings b ON a.room_id = b.room_id
                AND a.id < b.id
                AND a.start_ts < b.end_ts
                AND b.start_ts < a.end_ts;
```

▶ Two intervals overlap when each starts before the other ends. a.id < b.id avoids duplicate pairs.

49. Collapse consecutive numbers into ranges.

```
SELECT MIN(n) AS start_n, MAX(n) AS end_n
FROM (SELECT n, n - ROW_NUMBER() OVER (ORDER BY n) AS grp
      FROM nums) t
GROUP BY grp
ORDER BY start_n;   -- 1-3, 5-6, 9-9
```

▶ Same gaps-and-islands trick with numbers instead of dates.

50. Active subscriptions on each day.

```
SELECT c.d, COUNT(s.user_id) AS active
FROM calendar c
LEFT JOIN subs s ON c.d BETWEEN s.start_date AND COALESCE(s.end_date,
CURRENT_DATE)
GROUP BY c.d;
```

▶ Range join to a calendar. NULL end_date = still active.

Sagar

Classic problems

51. Numbers that appear at least 3 times in a row.

```
SELECT DISTINCT num
FROM (SELECT num,
             LEAD(num, 1) OVER (ORDER BY id) AS n1,
             LEAD(num, 2) OVER (ORDER BY id) AS n2
      FROM logs) t
WHERE num = n1 AND num = n2;
```

▶ LEAD looks ahead. Don't rely on id + 1 joins if ids can have gaps.

52. Days warmer than the previous day.

```
SELECT id
FROM (SELECT id, record_date, temp,
             LAG(temp) OVER (ORDER BY record_date) AS prev_temp,
             LAG(record_date) OVER (ORDER BY record_date) AS prev_date
      FROM weather) t
WHERE temp > prev_temp
  AND record_date = prev_date + INTERVAL '1' DAY;
```

▶ Check the previous row is really yesterday — dates can be missing.

53. Cancellation rate per day for unbanned users.

```
SELECT t.request_at AS day,
       ROUND(AVG(CASE WHEN t.status LIKE 'cancelled%' THEN 1.0 ELSE 0 END), 2) AS rate
FROM trips t
JOIN users c ON c.user_id = t.client_id AND c.banned = 'No'
JOIN users d ON d.user_id = t.driver_id AND d.banned = 'No'
GROUP BY t.request_at;
```

▶ AVG of a 1/0 flag = rate. Join users twice, once for client and once for driver.

54. Person with the most friends.

```
SELECT id, COUNT(*) AS num
FROM (SELECT requester_id AS id FROM friends
      UNION ALL
      SELECT accepter_id FROM friends) t
GROUP BY id
ORDER BY num DESC
LIMIT 1;
```

▶ A friendship counts for both people — stack both columns with UNION ALL.

Sagar

55. Swap every two adjacent seats.

```
SELECT CASE WHEN id % 2 = 1 AND id = (SELECT MAX(id) FROM seat) THEN id
            WHEN id % 2 = 1 THEN id + 1
            ELSE id - 1 END AS id,
       student
FROM seat
ORDER BY id;
```

▶ Odd ids move up, even ids move down; a last odd id stays put.

56. Label each node as Root, Inner or Leaf.

```
SELECT id,
  CASE WHEN parent_id IS NULL THEN 'Root'
       WHEN id IN (SELECT parent_id FROM tree WHERE parent_id IS NOT NULL) THEN
'Inner'
       ELSE 'Leaf' END AS type
FROM tree;
```

▶ The 'IS NOT NULL' inside matters — NULLs in IN-lists cause surprises.

57. Click-through rate per ad, without division errors.

```
SELECT ad_id,
       ROUND(100.0 * SUM(CASE WHEN action = 'click' THEN 1 ELSE 0 END)
            / NULLIF(SUM(CASE WHEN action IN ('click', 'view') THEN 1 ELSE 0 END), 0), 2)
AS ctr
FROM ad_events
GROUP BY ad_id;
```

▶ NULLIF(x, 0) turns a zero denominator into NULL instead of an error.

58. Product pairs most often bought together.

```
SELECT a.product_id AS p1, b.product_id AS p2, COUNT(*) AS times
FROM order_items a
JOIN order_items b ON a.order_id = b.order_id
                  AND a.product_id < b.product_id
GROUP BY a.product_id, b.product_id
ORDER BY times DESC
LIMIT 10;
```

▶ Market basket. a < b removes self-pairs and mirror duplicates.

59. Employee hierarchy: all reports under a manager, any level.

```
WITH RECURSIVE org AS (
  SELECT id, name, manager_id, 1 AS lvl FROM emp WHERE id = :mgr
  UNION ALL
  SELECT e.id, e.name, e.manager_id, o.lvl + 1
  FROM emp e JOIN org o ON e.manager_id = o.id)
SELECT * FROM org WHERE lvl > 1;
```

▶ Recursive CTE = anchor + recursive part. Add a depth limit if the data could have cycles.

Data engineering SQL

60. Upsert new and changed rows into a target (SCD Type 1).

```
MERGE INTO target t
USING staging s ON t.id = s.id
WHEN MATCHED AND (t.name <> s.name OR t.city <> s.city) THEN
  UPDATE SET name = s.name, city = s.city
WHEN NOT MATCHED THEN
  INSERT (id, name, city) VALUES (s.id, s.name, s.city);
```

▶ Deduplicate staging first — MERGE fails when two source rows match one target row. The change condition avoids rewriting unchanged rows.

61. SCD Type 2: keep history when a customer's city changes.

```
-- 1. close the current row for changed customers
UPDATE dim_customer d SET end_date = CURRENT_DATE, is_current = FALSE
WHERE d.is_current
  AND EXISTS (SELECT 1 FROM staging s WHERE s.id = d.id AND s.city <> d.city);

-- 2. insert a new current row for new + changed customers
INSERT INTO dim_customer (id, city, start_date, end_date, is_current)
SELECT s.id, s.city, CURRENT_DATE, NULL, TRUE
FROM staging s
LEFT JOIN dim_customer d ON d.id = s.id AND d.is_current
WHERE d.id IS NULL;
```

▶ Order matters: close first, then insert where no current row exists. Run both in one transaction.

Sagar

62. Incremental load: only rows changed since the last load.

```
INSERT INTO staging
SELECT *
FROM source
WHERE updated_at > (SELECT last_loaded FROM etl_watermark
                    WHERE table_name = 'source');

UPDATE etl_watermark
SET last_loaded = (SELECT MAX(updated_at) FROM staging)
WHERE table_name = 'source';
```

▶ Store the watermark in a control table. Use a small overlap window plus MERGE so late-arriving rows aren't missed.

63. Rows in source missing from target, and rows that changed.

```
-- missing or changed rows (full row compare)
SELECT * FROM src
EXCEPT
SELECT * FROM tgt;

-- only ids missing completely
SELECT s.id FROM src s
LEFT JOIN tgt t ON t.id = s.id
WHERE t.id IS NULL;
```

Sagar

▶ EXCEPT compares every column, and treats NULLs as equal, unlike '='.

64. Reconcile row counts between source and target per load date.

```
SELECT COALESCE(s.load_date, t.load_date) AS load_date,
       s.cnt AS src_cnt, t.cnt AS tgt_cnt
FROM (SELECT load_date, COUNT(*) AS cnt FROM src GROUP BY load_date) s
FULL OUTER JOIN
     (SELECT load_date, COUNT(*) AS cnt FROM tgt GROUP BY load_date) t
  ON s.load_date = t.load_date
WHERE s.cnt IS DISTINCT FROM t.cnt;
```

▶ FULL JOIN catches dates present on only one side. Real reconciliation also compares SUMs of key amounts.

65. Check staging for duplicate primary keys before loading.

```
SELECT order_id, COUNT(*) AS cnt
FROM staging
GROUP BY order_id
HAVING COUNT(*) > 1;
```

▶ Run as a blocking check in the pipeline: if this returns rows, fail the load.

66. Null-count report for key columns in one pass.

```
SELECT COUNT(*) AS total,
       SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) AS null_email,
       SUM(CASE WHEN phone IS NULL THEN 1 ELSE 0 END) AS null_phone,
       SUM(CASE WHEN city  IS NULL THEN 1 ELSE 0 END) AS null_city
FROM customers;
```

▶ One scan instead of one query per column. Add thresholds and alert on them.

67. Latest row per key using QUALIFY.

```
SELECT *
FROM events
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1;
```

▶ QUALIFY filters on window results without a subquery (Databricks/Spark SQL, Snowflake, BigQuery).

68. Explode an array column into rows.

```
-- Spark / Hive
SELECT order_id, item
FROM orders LATERAL VIEW EXPLODE(items) t AS item;

-- BigQuery / Postgres
SELECT order_id, item
FROM orders, UNNEST(items) AS item;
```

▶ Know your dialect. LATERAL VIEW OUTER EXPLODE keeps rows with empty arrays.

69. Why does this query scan the whole table? Fix it.

```
SELECT *
FROM events
WHERE event_date >= DATE '2026-01-01'
  AND event_date <  DATE '2027-01-01';
```

▶ A function on the partition column blocks partition pruning. Filter the raw column with a range.

Concepts & output traps

70. Logical order of execution of a SELECT.

```
FROM / JOIN → WHERE → GROUP BY → HAVING
→ SELECT → DISTINCT → ORDER BY → LIMIT
```

▶ Explains why a SELECT alias can't be used in WHERE, but can be used in ORDER BY.

Sagar

71. WHERE vs HAVING.

```
-- WHERE filters rows before grouping
-- HAVING filters groups after aggregation
SELECT dept_id, AVG(salary)
FROM emp
WHERE hire_date >= DATE '2020-01-01'
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

▶ Filter in WHERE whenever possible — less data reaches the GROUP BY.

72. Trap: COUNT(*) vs COUNT(col) vs COUNT(DISTINCT col).

```
COUNT(*)            → 4
COUNT(col)          → 3
COUNT(DISTINCT col) → 2
```

▶ COUNT(col) skips NULLs; DISTINCT also removes duplicates.

73. UNION vs UNION ALL.

```
-- UNION     → removes duplicates (extra sort/shuffle)
-- UNION ALL → keeps everything, faster
```

▶ Default to UNION ALL unless you actually need deduplication.

74. Trap: why does WHERE city <> 'Delhi' miss some rows?

```
SELECT * FROM customers
WHERE city <> 'Delhi' OR city IS NULL;
```

▶ NULL <> 'Delhi' is UNKNOWN, not TRUE, so those rows are filtered out.

75. Trap: why does NOT IN return no rows?

```
SELECT * FROM a
WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.id = a.id);
```

▶ x NOT IN (1, NULL) means x<>1 AND x<>NULL → UNKNOWN for every row.

76. DELETE vs TRUNCATE vs DROP.

```
DELETE   → removes rows (can use WHERE), logged, slower
TRUNCATE → removes all rows fast, keeps the table structure
DROP     → removes the table and its structure
```

▶ Rollback behaviour depends on the database — say that rather than guessing.

Sagar

77. CTE vs subquery vs temp table.

```
CTE        → readable, named step inside one query
Subquery   → inline, fine for small logic
Temp table → materialized once, reused across queries
```

▶ CTEs aren't always materialized — some engines recompute them each time they're referenced.

78. A query got slow. How do you debug it?

```
1. EXPLAIN / query plan: full scans, join type, spills
2. Filter early, select only needed columns
3. Partition pruning / indexes on filter and join keys
4. Check data skew and duplicate join keys
5. Replace correlated subqueries with joins or windows
```

▶ Talk through the plan first. Guessing fixes without reading the plan is the red flag they look for.

— Sagar

Sagar